



Functional Specification

Interactive Leaflet map in ONE Portal

Version 1.0

DOVER FUELING SOLUTIONS CONFIDENTIAL INFORMATION, SUBJECT TO NDA.

THE DRAWINGS, TECHNICAL DATA, AND ALL OTHER INFORMATION SET FORTH IN THIS DOCUMENT COMPRISE TRADE SECRETS AND/OR OTHER CONFIDENTIAL INFORMATION THAT IS THE PROPERTY OF DOVER FUELING SOLUTIONS, UK LTD. AND/OR ITS AFFILIATE(S) AND MAY BE SUBJECT TO COPYRIGHT OR OTHER PROTECTION. ACCESS TO AND/OR USE OF SUCH CONFIDENTIAL INFORMATION IS STRICTLY CONTROLLED AND LIMITED TO THE PURPOSE(S) FOR WHICH IT IS OR WAS PROVIDED, AND NO RECIPIENT IS PERMITTED TO USE, COPY, OR OTHERWISE DISCLOSE OR TRANSMIT SUCH INFORMATION, EXCEPT AS EXPRESSLY SET FORTH IN THE APPLICABLE NON-DISCLOSURE AGREEMENT(S), OR AS OTHERWISE AUTHORIZED IN WRITING BY, OR ON BEHALF OF, DOVER FUELING SOLUTIONS UK LTD. AND/OR ITS AFFILIATE(S). ALL RIGHTS RESERVED.





EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

TABLE OF CONTENTS

1. RECORD OF CHANGES	4
2. INTRODUCTION	5
2.1 ABBREVIATIONS	5
3. LEAFLET MAP FUNCTIONALITIES	6
3.1 TOGGING BETWEEN TAB AND MAP VIEW	6
3.2 DISPLAYING A COUCOM ON THE MAP	7
3.3 INTERPRETING THE MAP	8
3.4 MARKER POPUPS	9
3.5 MARKER CLUSTERING	10
3.6 FULLSCREEN	11
3.7 ZOOM FUNCTIONS	11
3.8 MAP LEGEND	11
3.9 TILE PROVIDERS	12
3.10 TILE PROVIDER FALLBACK, BOUNDARIES AND ZOOM LIMITS	15
4. STATION CONFIGURATOR	17
4.1 VIEW ONLY MAP INSIDE STATION DETAIL FRAME	17
17	
4.2 CREATE STATION AUTOMATED GEOCODING	17
4.3 MARKER DRAGGING	18
4.4 DIFFERENT UPDATE FORM BASED ON EDIT RIGHTS	18
5. MANUAL: STATION BATCH GEOCODING SCRIPT	19
5.1 PREPARING THE DATA (CSV FILE)	19
5.2 SETTING UP THE PYTHON ENVIRONMENT	20
5.3 RUNNING THE SCRIPT	21
5.4 COLLECTING MISSED STATIONS	22
5.5 IMPORTING COORDINATES BACK TO THE DATABASE	23
6. LEAFLET MANUAL	25
6.1 MAP COMPONENT	26
6.1.1 MAP DATA INTERFACE	26
6.1.2 MAP INITIALIZATION	26
6.1.3 MAP STYLING	28
6.1.4 USING HTML WITH LEAFLET	28
6.1.5 TOGGLE MAP BUTTON	29
6.2 ADDING CONTROLS	30



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

6.2.1	IMPLEMENTING FULLSCREEN CONTROL.....	30
6.2.2	ADDING A CONTROL FOR CHANGING TILE LAYERS.....	31
6.2.3	IMPLEMENTING A CUSTOM TOGGLE CLUSTERING BUTTON.....	32
6.2.4	ADDING THE LEGEND CONTROL.....	34
6.3	CREATE MARKERS METHOD	36
6.4	COUPLING CUSTOM COMPONENTS TO POPUPS.....	40
6.5	IMPLEMENTING DONUT CLUSTERS (CREATEDONUTCLUSTER).....	42
6.6	TILE SERVICE FAILSAFE (SETTILEFAILSAFE)	45
7.	POPUP COMPONENT.....	46
8.	TREE COMPONENT.....	47
8.1	WALKING THE TREE.....	47
8.2	RECURSIVE TREE SEARCHING (SEARCHTREENODE).....	49
8.3	HANDLING USER SELECTION CHANGES (ONSELECTIONCHANGE).....	51
8.4	REFRESH TIMER (M_REFRESHTIMERSUBSCRIPTION)	52



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

1. RECORD OF CHANGES

Version 1.0	April 20, 2026	S.Backx
	• Original document.	
Version 2.0	May 20, 2026	S.Backx
	• Updated with Geocoding manual • Updated with Leaflet manual	



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

2. INTRODUCTION

We are adding an interactive Leaflet dashboard map to the existing alerts module in ONE Portal. This map allows clients to display all stations within a CoucCom network. To build this within ONE Portal, we will use the open-source Leaflet library and translate station address data into exact geographical coordinates for accurate positioning.

This new visual map dashboard will complement the current tab view. It will provide clients with a high-level visual overview of all CouCom stations and their live status. This document outlines all map functionalities and provides a manual for the geocoding script and leaflet library.

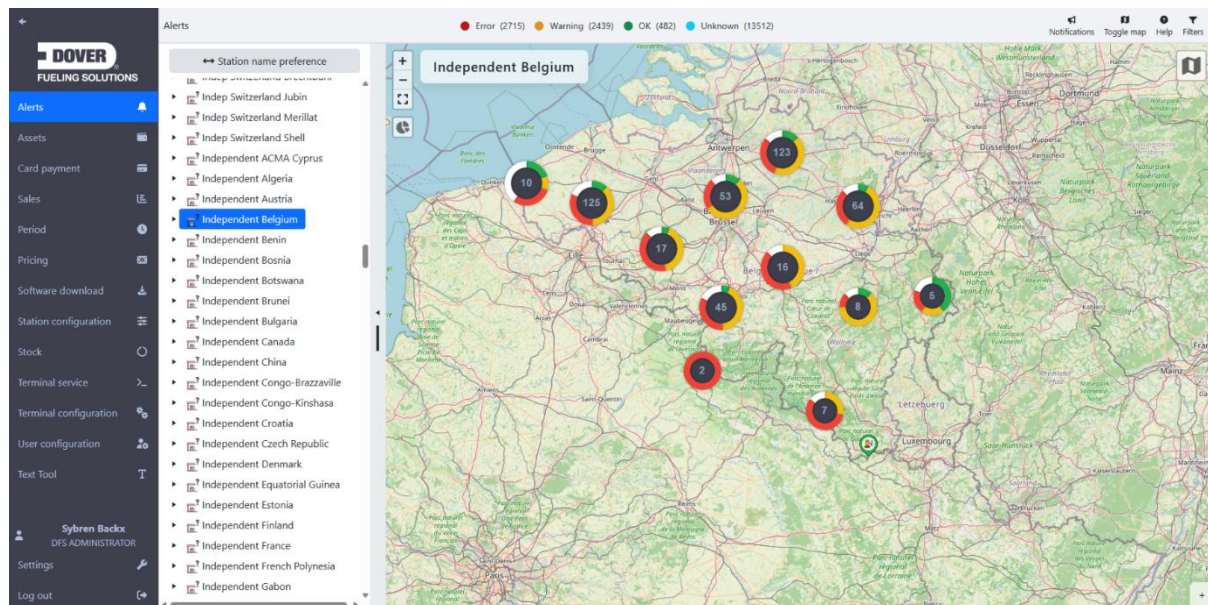
2.1 ABBREVIATIONS

OP	ONE Portal
PM	Petrol Manager
SC	Station Configurator

3. LEAFLET MAP FUNCTIONALITIES

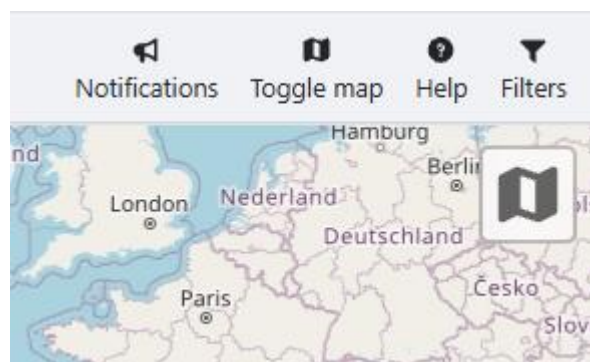
The Leaflet map extends the existing ONE Portal alerts module as a new component inside the main body component. The image below demonstrates the map interface when a user selects the Independent Belgium CouCom network.

3.1 TOGGLING BETWEEN TAB AND MAP VIEW



1 Leaflet alerts map

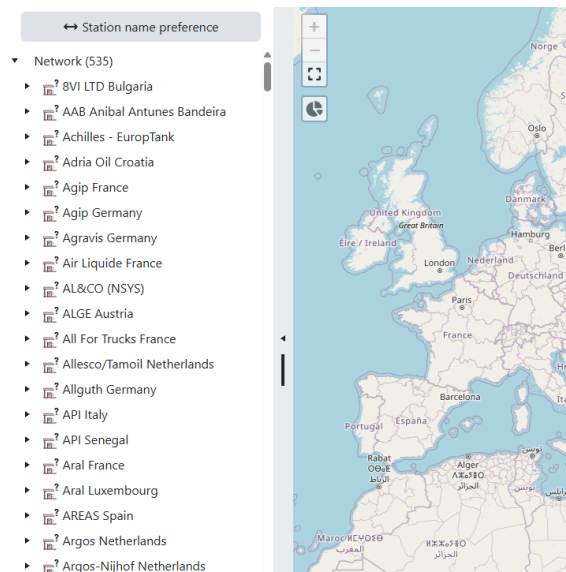
To view the map, click the “Toggle map” button in the top right corner. This button instantly switches the interface between the original tab view and the new map view.



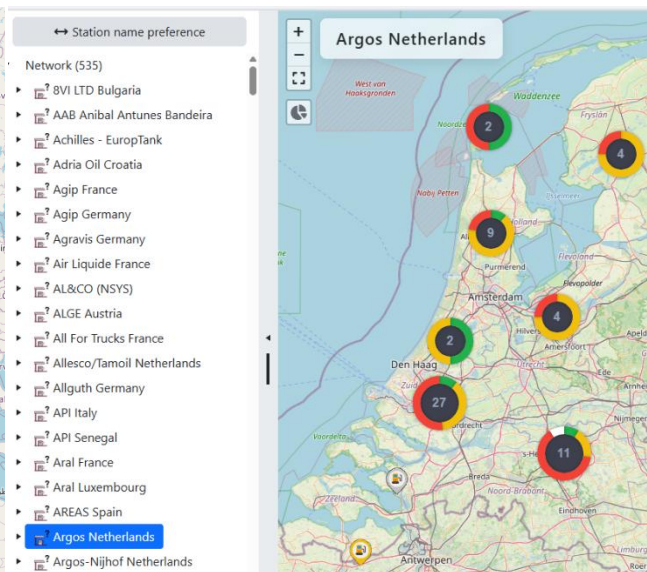
2 Map toggle button

3.2 DISPLAYING A COUCOM ON THE MAP

The map integrates directly with the existing navigation tree on the left. Users display CouCom data by clicking a specific CouCom node within this tree. The first image below shows the default view with no selection, while the second image displays the map after selecting the “Argos Netherlands” CouCom node.



3 Default map view nothing selected



4 Map view with Coucom selected

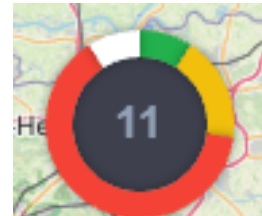
3.3 INTERPRETING THE MAP

Clusters

Selecting a CouCom displays two types of icons. The first type is a pie chart representing a cluster (a group of stations). The central number shows the total station count within that cluster.

The colored segments display the status distribution of these stations:

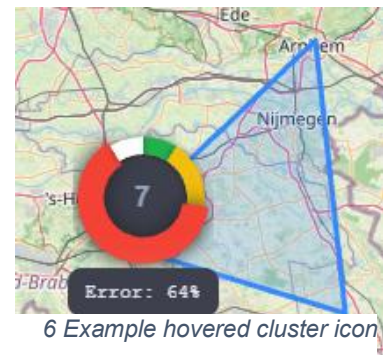
- Green status "OK",
- Yellow status "Warning",
- Red status "Error",
- White status "Undefined".



5 Example cluster icon

Hovering over a cluster draws a blue polygon on the map to outline the geographic region of the contained stations.

Hovering over a specific colored segment changes the central number to display only the station count for that status. The map also displays a percentage at the bottom to show that segment's share of the total cluster (e.g., 7 stations representing 64% in "Error" status).



6 Example hovered cluster icon

Marker Icons

The map uses the following marker icon in four distinct colors to represent individual stations:

-  : A station with the status "OK".
-  : A station with the status "Warning".
-  : A station with the status "Error".
-  : A station with the status "Undefined".

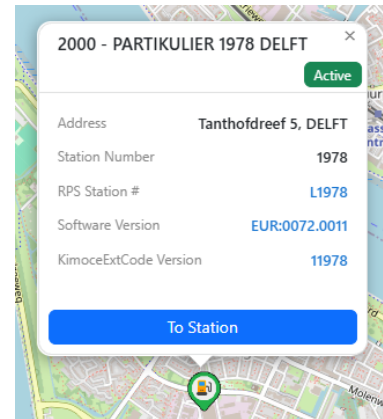
3.4 MARKER POPUPS

Popup Card

The station markers on the map are clickable. When clicking a marker, a popup card will appear above that marker. The popup card contains information about the station.

Popup card content:

- Station Name
- Address
- Station RPS number
- KimoceExtCode
- Station state badge
- Station Number
- Software version

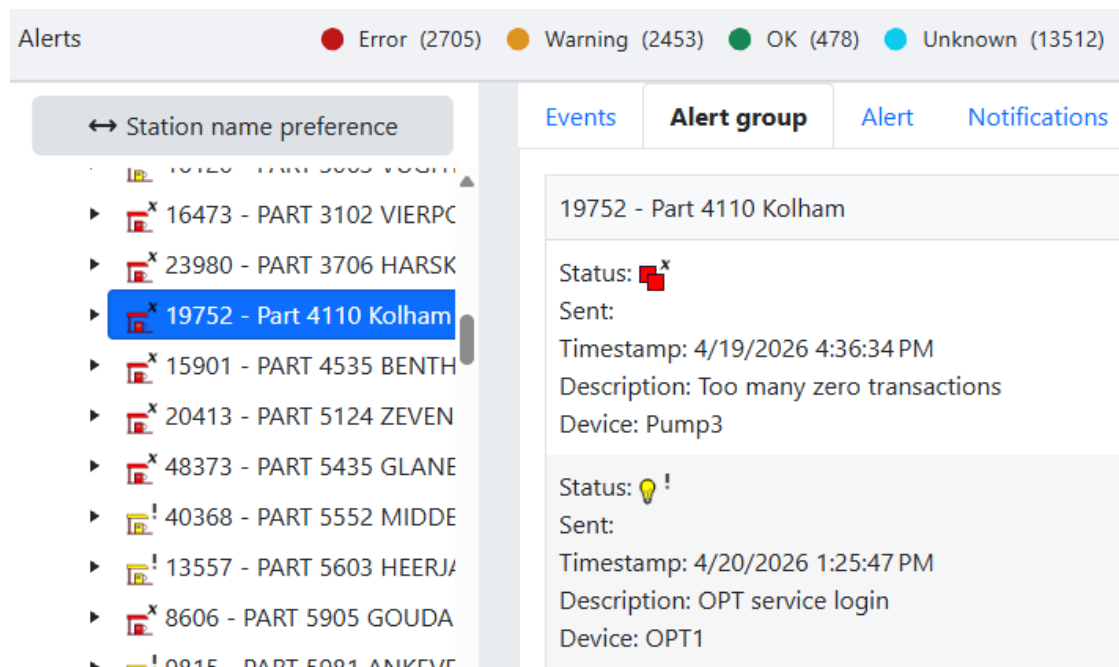


7 Popup card

A dynamic status badge in the top-right corner mirrors the station's state. This badge changes color to green, yellow, red, or white depending on the current station state.

Drill Down Button

A blue "To Station" button sits at the bottom of the popup card. Clicking this button triggers a recursive search in the left navigation tree to locate that specific station. Once found, the interface automatically switches back to the tab view, allowing users to drill down instantly.



8 Screen after pressing drill down button

3.5 MARKER CLUSTERING

The two images below compare the "Independent Netherlands" CouCom across two map views. Displaying this network's 900 stations simultaneously creates severe visual clutter, making individual markers impossible to select.

To maintain high visibility, the map implements marker clustering to group geographically close stations based on screen proximity.

The second image demonstrates active clustering, which groups stations into clear pie charts. Users drill down into these groups by clicking a cluster. This action automatically zooms the map and recalculates the clusters. Repeated clicks continuously shrink the clusters until individual station markers become visible.



9 Map without marker clustering enabled

Toggle Button

Users enable or disable clustering using the bottom button in the top-left corner of the map. The button icon toggles between a pie chart and a standard marker to indicate the active mode. This flexibility allows users to disable clustering for smaller networks, such as those with only 40 stations, where clustering is unnecessary.



10 Map with marker clustering enabled

Performance

Marker clustering significantly improves map performance. Concurrently rendering 900 stations alongside their popup component references creates a heavy processing load for ONE Portal. Clustering partially resolves this bottleneck.

To ensure smooth user experience, the map implements two additional optimization techniques:

- **Lazy-Loading Popups:** Instead of instantiating and storing popups for every marker simultaneously, the system creates the popup component only upon a user click. Deselecting a popup instantly unbinds and destroys it to free up memory.
- **Viewport-Based Loading:** The map filters out markers and clusters that fall outside the user's visible screen area. The system dynamically loads assets only when they enter the active viewport.

3.6 FULLSCREEN

The map includes a fullscreen mode designed for large control room monitors. This mode expands the map view to display an entire CouCom network simultaneously. Users activate this feature via the third button from the top in the top-left corner of the map.

3.7 ZOOM FUNCTIONS

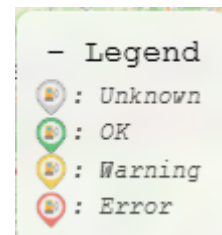
Users zoom the map using the mouse scroll wheel or by clicking the "+" and "-" buttons located in the top-left corner.



11 Map controls

3.8 MAP LEGEND

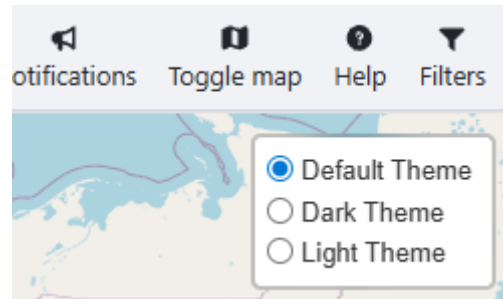
A status legend sits in the bottom-right corner of the map to define the station marker color codes. By default, a small "+" icon hides this legend to maximize map visibility; clicking it expands the full color-code key.



12 Map legend

3.9 TILE PROVIDERS

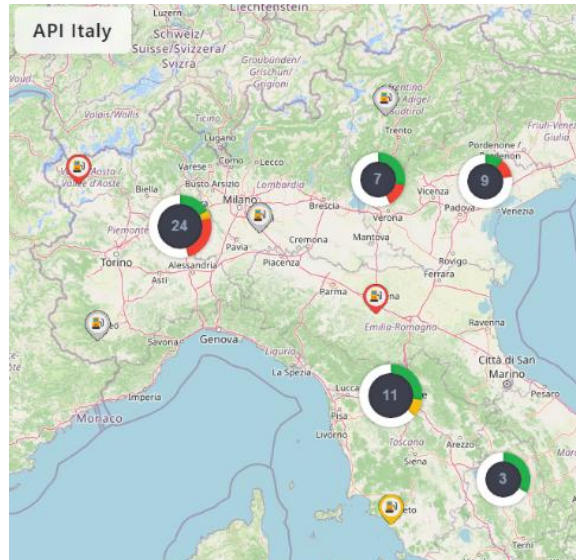
A tile provider is a remote server system that delivers a grid of seamless, pre-rendered square images (typically 256x256 pixels) or vector data packages to the browser. Instead of loading the entire world map at once—which would crash the application—the Leaflet library dynamically requests only the specific "tiles" needed for the user's current coordinates and zoom level. As the user pans or zooms, the map requests new tiles on the fly to fill the screen smoothly. This provider ultimately dictates the aesthetic style, language, borders, and geographical details of the background map.



13 Tile provider controls

The ONE Portal map integrates 4 distinct open-source, free-of-cost tile providers to give users different viewing options (e.g., standard street view, high-contrast dark mode, or terrain view). Users switch the map style by hovering over the layer control button in the top-right corner and selecting their preferred provider.

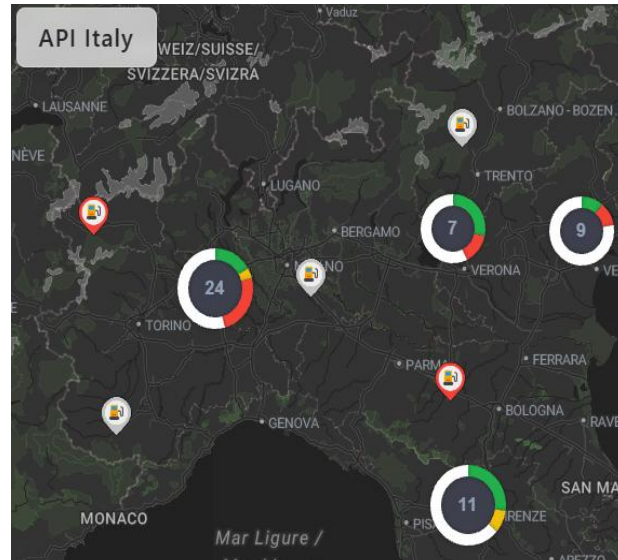
Default Theme / OpenStreetMap Default Provider



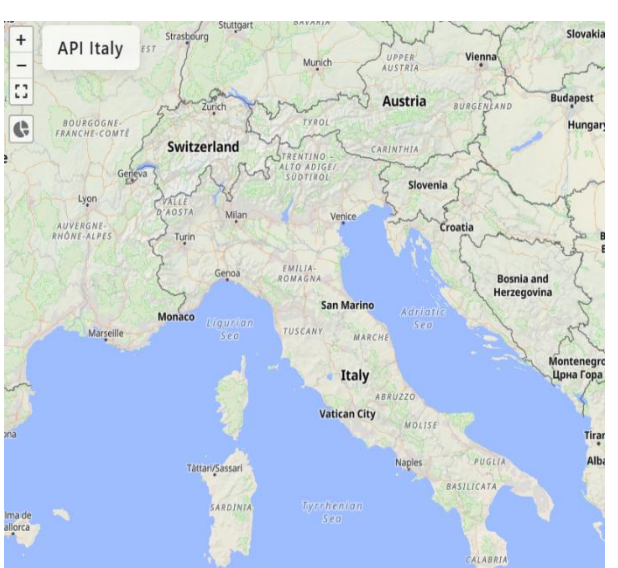
Light Theme / CartoDB_Positron Provider



Dark Theme / Stadia_AlidadeSmoothDark Provider



Fallback Theme / OpenFreeMap Provider





EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

Tile provider policies

The following section is straight from the Nominatim OpenStreetMap policy page. It describes the requirements that need to be met in order to make use of their tile service and public API. Not following these guidelines, might result in access being blocked and receiving the following screen.



14 Account suspension screen

Requirements

- No heavy uses (an absolute **maximum of 1 request per second**).
- Provide a valid **HTTP Referer** or **User-Agent** identifying the application (stock User-Agents as set by http libraries will not do).
- Clearly display [attribution](#) as suitable for your medium.
- Data is provided under the [ODbL](#) license which requires to share alike (although small extractions are likely to be covered by fair usage / fair dealing).

Websites and apps

Use that is directly triggered by the end-user (for example, user searches for something) is ok, provided that your number of users is moderate. Note that the usage limits above apply **per website/application**: the sum of traffic by all your users should not exceed the limits.

Apps must make sure that they can switch the service at our request at any time (in particular, switching should be possible without requiring a software update). If at all possible, **set up a proxy** and also enable caching of requests.

Note: periodic requests from apps are considered bulk geocoding and as such are strongly discouraged. It may be okay if your app has very few users and applies appropriate caching of results. Make sure you stay well below the API usage limits.



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

Bulk Geocoding

As a general rule, bulk geocoding of larger amounts of data is **not** encouraged. If you have regular geocoding tasks, please, look into alternatives below. Smaller one-time bulk tasks may be permissible, if these additional rules are followed

- limit your requests to a **single thread**
- limited to 1 machine only, **no distributed scripts** (including multiple Amazon EC2 instances or similar)
- Results **must be cached** on your side. Clients sending repeatedly the same query may be classified as faulty and blocked.

Unacceptable use

The following uses are strictly forbidden and will get you banned:

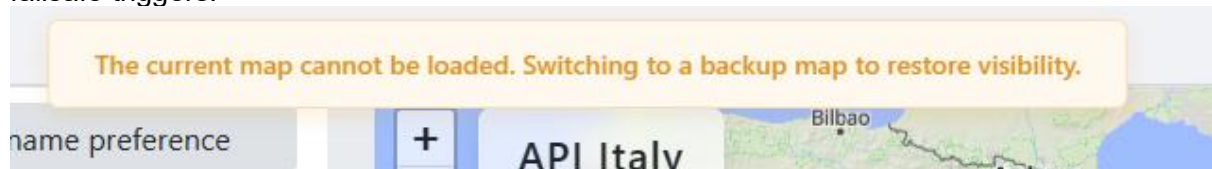
- **Auto-complete search** This is not yet supported by Nominatim and you must not implement such a service on the client side using the API.
- **Systematic queries** This includes reverse queries in a grid, searching for complete lists of postcodes, towns etc. and downloading all POIs in an area. If you need complete sets of data, get it from the [OSM planet](#) or an extract.
- **Scraping of details** The details page is there for debugging only and may not be downloaded automatically.

3.10 TILE PROVIDER FALLBACK, BOUNDARIES AND ZOOM LIMITS

To limit tile usage and prevent our access being revoked, a caching check is put into place. This measure makes sure that users load map tiles from their browser cache instead of continuously requesting the same tiles.

MAP FALLBACK TILE PROVIDER

If OpenStreetMap becomes unreachable or fails to serve tiles, the map automatically triggers a failsafe mechanism. The system switches to the OpenFreeMap tile provider as its primary backup to ensure uninterrupted map visibility. The following warning gets shown when the failsafe triggers.



15 Tile Provider Fallback Alert

MAP BOUNDARIES

Selecting a CouCom triggers an automated camera flight to the geographic boundaries of that specific network.

Originally, the system calculated the median coordinates of all network stations to center the map. However, locking the view to a static median calculation broke marker selection animations. Clicking an individual station marker requires the map to pan and center on that marker, which a fixed bounding box prevented.

To resolve this, the system now dynamically calculates the outer bounding box corners and applies a margin of padding around them. This padding guarantees that all stations fit perfectly on screen without obstructing pan animations. Furthermore, these bounding limits restrict unnecessary scrolling, which minimizes traffic to the tile provider by preventing the map from loading redundant tiles.

ZOOM CONSTRAINTS AND LOCKING

When no network is selected, the map locks to a default zoom level and disables all panning and navigation. This restriction prevents unnecessary zooming and stops the system from loading redundant map tiles.

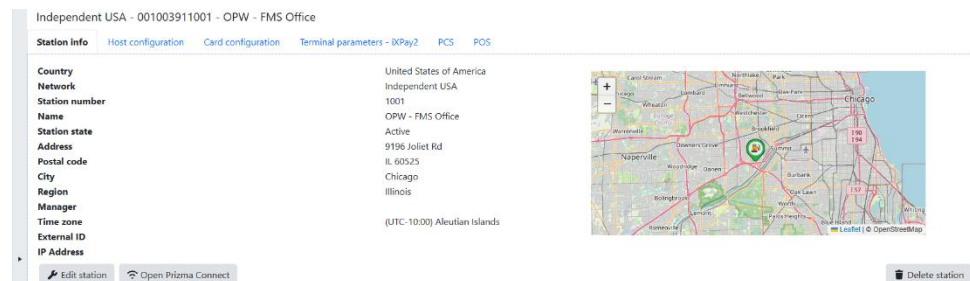
Once a user selects a CouCom and the camera finishes panning to the location, the system unlocks the map and applies zoom limits. These configured minimum and maximum constraints prevent users from zooming too far out or losing focus on the relevant network area.

4. STATION CONFIGURATOR

4.1 VIEW ONLY MAP INSIDE STATION DETAIL FRAME

The station details frame within the Station Configurator module now includes a view-only map. This component provides clients with an immediate visual reference of their station's physical location.

The interface disables all map panning and clicking interactions. Users can only adjust the view by using the map's zoom buttons.



16 View Only Map In SC

4.2 CREATE STATION AUTOMATED GEOCODING

The station creation and update workflows within the ONE Portal Station Configurator module now automatically generate coordinates. The system integrates the open-source Nominatim API to handle this geolocation lookup.

When a user submits a creation or update form, the system automatically builds a search query using three specific fields:

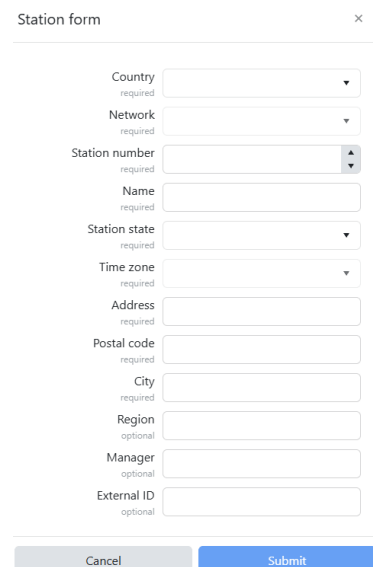
- **Address**
- **Postal Code**
- **Country Name**

The application sends this query to the Nominatim API, retrieves the matching latitude and longitude, and saves these coordinates directly into the Petrol Manager (PM) configuration database.

To optimize API usage, the system applies distinct execution rules based on the user's action:

Creating a Station: The system always executes the Nominatim API call and populates the coordinates.

Updating a Station: The system triggers the API call only if the user modifies the address, postal code, or country fields. If these fields remain unchanged, the lookup is skipped.



The screenshot shows the 'Station form' with the following fields:

- Country (required)
- Network (required)
- Station number (required)
- Name (required)
- Station state (required)
- Time zone (required)
- Address (required)
- Postal code (required)
- City (required)
- Region (optional)
- Manager (optional)
- External ID (optional)

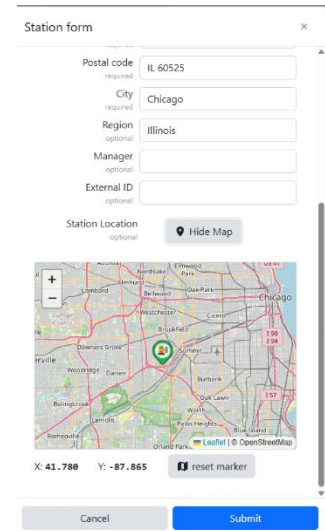
Buttons: Cancel, Submit

17 Create Station Form

4.3 MARKER DRAGGING

The update station form features a "Show map" toggle button. Activating this option displays a map with a single, draggable station marker that allows users to manually adjust the station's location. To safeguard against accidental movements, the interface includes a "Reset marker" button that instantly snaps the marker back to its original saved position.

Upon form submission, the system saves the exact coordinates where the user dropped the marker. If a user modifies both the marker position and the textual address fields simultaneously, the dragged marker coordinates take absolute priority over the automated address lookup.



18 Update Station Form

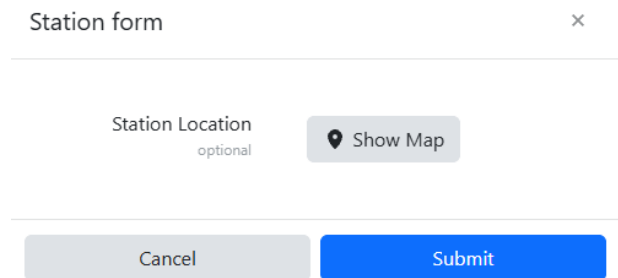
4.4 DIFFERENT UPDATE FORM BASED ON EDIT RIGHTS

The update form dynamically adapts based on the station type (Petrol Manager vs. non-PM) and the user's specific access rights.

Previously, the system hid the edit button from users lacking update permissions. The interface now exposes the edit button to all users but restricts field visibility conditionally:

PM station: Access and edit the entire configuration form and map options.

Non-PM station: Access the map interface exclusively to use the marker-dragging option, while all text fields remain locked.



19 Update Station Form Non-PM Station

5. MANUAL: STATION BATCH GEOCODING SCRIPT

This manual guides you through preparing your station data, configuring the Python environment, running the automated geocoding script, and re-importing the calculated coordinates back into the database.

5.1 PREPARING THE DATA (CSV FILE)

The script expects a csv input file containing your station data. You can create one by creating a text file and renaming the suffix to “.csv”

REQUIRED INPUT FILENAME

File Name: “stations.csv”

Location: Place this file in the same folder where the Python script is saved.

REQUIRED COLUMNS

Your file must contain commas as delimiters and include headers that match these names:

- StationId
- StationName
- Address
- Zip
- City
- CouComName

You can check the delimiter when you open your csv with a simple text editor like notepad.

GETTING THE COLUMNS

You can get the station data by copy pasting the columns over to the csv using the following SQL script.

```
SELECT
    s.[StationId]
    ,s.[StationName]
    ,s.[Address]
    ,s.[Zip]
    ,s.[City]
    ,c.[CouComName]
FROM [PetrolManagerConfiguration].[dbo].[Station] s
INNER JOIN [PetrolManagerConfiguration].[dbo].[CouCom] c
    ON s.[CouComId] = c.[CouComId];
```

5.2 SETTING UP THE PYTHON ENVIRONMENT

STEP 1: INSTALL PYTHON

1. Download Python 3.10 or higher from the [Official Python Website](https://www.python.org/).
2. Run the installer and check the box that says **"Add Python to PATH"** before clicking Install.

STEP 2: INSTALL REQUIRED LIBRARIES

Open your terminal or Command Prompt and run this command to install the script dependencies:

```
- pip install pandas requests tqdm
```

WHAT THESE LIBRARIES ARE FOR:

pandas: This library handles data manipulation and analysis. The script uses it to open your stations.csv file, read the rows, check the cache file, and save the final geocoded results into a new CSV file.

requests: This library allows Python to send HTTP requests over the internet. The script uses it to safely communicate with the external Nominatim (OpenStreetMap) API to fetch coordinates for your addresses.

tqdm: This library generates, real-time progress bars. It provides the visual loading bar in your terminal so you can see exactly how many stations have been processed and estimate the remaining time.



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

5.3 RUNNING THE SCRIPT

1. Make sure your script file 'StationConverter.py' and stations.csv in the same directory.
2. Open your terminal or Command Prompt, navigate to that folder, and run:

```
- python geocoder.py
```

3. A progress bar will show the translation progress.

HOW THE SCRIPT WORKS

API Compliance: The script limits calls to **1 per second** to abide by OpenStreetMap's terms of service.

Failover Logic: If the full address search fails, the script automatically retries with three fallback variations:

1. *Full Search:* Address + ZIP + Country.
2. *Fallback 1:* Stripped street name + ZIP + Country.
3. *Fallback 2:* Station Name + ZIP + City + Country.
4. *Fallback 3:* ZIP + Country.
5. *Fallback 4:* Country.

Caching System: The script creates a local file named "geocode_cache.csv". If you stop the script and restart it, it reads this file and skips already-translated addresses.

OUTPUT

When complete, the script creates a "stations_geocoded.csv" file in the directory where the script ran. This file contains your original columns with the "lat" and "lon" columns appended to it.

5.4 COLLECTING MISSED STATIONS

Some station addresses might fail all fallback loops if they contain spelling errors or invalid zip codes. To get these exceptions without scrolling through the main output file, you can run a secondary script.

HOW TO RUN THE EXCEPTION SCRIPT

1. Save the secondary script as “MissedStations.py” in the same directory.
2. Change the INPUT_CSV variable inside the script to match your output file name: “stations_geocoded.csv”.
3. Open your terminal or Command Prompt and run:

```
- python MissedStations.py
```

OUTPUT

When executed, this script filters your data and creates a new “missed_stations” csv. It looks for records where the coordinates column is empty and puts the record in a new data frame. By running this, you will have a log of all stations where no coordinates can be found through Nominatim.

5.5 IMPORTING COORDINATES BACK TO THE DATABASE

STEP 1: CREATE A TEMPORARY STAGING TABLE

Do not import directly into your live production table. First, create a temporary staging table to hold the CSV contents safely. Open SQL Server Management Studio and run:

```
CREATE TABLE TempGeocodedStations (  
    stationId INT,  
    station VARCHAR(255),  
    address VARCHAR(255),  
    zip VARCHAR(50),  
    city VARCHAR(100),  
    country VARCHAR(100),  
    lat VARCHAR (15),  
    lon VARCHAR (15)  
);
```

STEP 2: IMPORT THE OUTPUT CSV TO STAGING

1. Right-click your database, select **Tasks**, then select **Import Flat File**.
2. Set the Data Source to **Flat File Source** and browse to select "stations_geocoded.csv".
3. Set the Destination to **Microsoft OLE DB Provider for SQL Server**.
4. Map the source columns to your new "TempGeocodedStations" table and complete the import wizard.

STEP 3: RUN THE SQL UPDATE SCRIPT

Execute this UPDATE query to move the new lat and lon fields into your active stations configuration table.

```
UPDATE prod_station  
SET  
    prod_station.[GpsLatitude] = csv_data.[lat],  
    prod_station.[GpsLongitude] = csv_data.[lon]  
FROM [PetrolManagerConfiguration].[dbo].[Station] prod_station  
INNER JOIN TempGeocodedStations csv_data  
    ON prod_station.[StationId] = csv_data.[stationId]  
WHERE csv_data.[lat] IS NOT NULL AND csv_data.[lon] IS NOT NULL;
```



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

STEP 4: CLEAN UP STAGING

Once the coordinates are in your desired table, you can drop the temporary table.

6. LEAFLET MANUAL

This manual explains the map configuration within the alerts module and its core features. This manual is a good starting point to implement Leaflet map components across other modules.

Leaflet documentation:

[Leaflet docs](#)

Tile Providers:

[Usefull demo to choose providers](#)

Plugin imports:

[leaflet.fullscreen](#)

[leaflet.markercluster](#)

[@kalisio/leaflet.donutcluster](#)

[@maplibre/maplibre-gl-leaflet](#)

Typescript types:

[leaflet types](#)

[leaflet.markercluster types](#)

The kalisio/leaflet.donutcluster plugin does not have a npm package available at the moment for types. In the map component under the types folder, there is a donutcluster-typings.d type declaration file. This file translates the javascript plugin so it can be used with typescript.

6.1 MAP COMPONENT

6.1.1 MAP DATA INTERFACE

```
export interface MapData {  
    /** Index of a station for the searchtree method*/  
    Index: number;  
    /** The alertlevel of a station*/  
    AlertLevel: number;  
    /** Station city for display inside marker popup*/  
    City: string;  
    /** Station address for display inside marker popup*/  
    Address: string;  
    /** Station GpsLatitude for calculating marker positions and map  
    boundaries*/  
    GpsLatitude: string;  
    /** Station GpsLongitude for calculating marker positions and map  
    boundaries*/  
    GpsLongitude: string;  
    /** Station RpsStationNumber for display inside marker popup*/  
    RpsStationNumber: string;  
    /** Station KimoceExtCode for display inside marker popup*/  
    KimoceExtCode: string;  
    /** Station SoftwareVersion for display inside marker popup*/  
    SoftwareVersion: string;  
    /** Station StationId not used*/  
    StationId: number;  
    /** Station StationName for display inside marker popup*/  
    StationName: string;  
    /** Station StationNumber for display inside marker popup*/  
    StationNumber: string;  
    /** Station StationState for setting the kendo badge inside marker popup*/  
    StationState: number;  
    /** Station StationStateName for display inside marker popup*/  
    StationStateName: string;  
}
```

6.1.2 MAP INITIALIZATION

To initialize a basic Leaflet map, follow these 4 steps:

Add the HTML Container

Create a **div** element with a unique **ID** to hold the map.

```
<div id="map"></div>
```

Add the HTML Container

In your css, give the map a **width** and **height**.

```
#map {  
  min-height: 50vh;  
  width: 50%;  
}
```

Define a Tile Layer

Define **OpenStreetMap** as the background tile layer provider. You can copy the code for providers [here](#).

```
public m_osm = L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {  
  maxZoom: 19,  
  attribution: '© OpenStreetMap'  
});
```

Initialize the Map Object

Instantiate the map inside your component using the **container ID**. A **center** starting point and starting **zoom level** are required to initiate the map.

```
this.m_map = L.map('map', {  
  center: [51.505, -0.09],  
  zoom: 13  
  layers: [this.m_osm]  
});
```

6.1.3 MAP STYLING

Standard Approach

Normally, Leaflet is set up by linking the stylesheet via a `<link>` tag in the HTML `<head>`. However, doing this creates an issue: the external stylesheet takes priority over your project's local SCSS files. Because of this loading order, any custom styles or overrides you write in your SCSS are ignored.

Solution

To fix this, the Leaflet CSS was copied and integrated directly into the third-party-shared module under custom themes named `leaflet.scss`. Importing the styles locally enables you to customize the default leaflet styling

Plugin style sheets

Some plugins also require a style sheet to be linked. For example here for the fullscreen and maplibreGL plugins.

```
<head>
  <link rel="stylesheet"
href="https://unpkg.com/leaflet.fullscreen/dist/Control.FullScreen.css" />
  <link href="https://unpkg.com/maplibre-gl/dist/maplibre-gl.css" rel="stylesheet"
/>
</head>
```

The markercluster and donutcluster plugin style sheets are all combined in the third-party-shared scss file to be able to alter the default styles.

6.1.4 USING HTML WITH LEAFLET

In a leaflet map, **you cannot use HTML template markup**. The map is not a collection of static HTML elements that you can declare ahead of time.

Once initialized, Leaflet takes complete control over this container. Everything you see on the map is dynamically generated via TypeScript and injected directly into this div at runtime.

Using components

Because Leaflet handles its own DOM manipulation, standard framework features (like Angular templates, structural directives, or data-binding) do not work inside map elements by default.

To use custom UI components inside map elements, you must programmatically instantiate and attach them. For an example on how to use components dynamically and link it to your map, see **Section 6.X: Creating and Coupling Popup Components**.



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

6.1.5 TOGGLE MAP BUTTON

The map view can be toggled open or closed in two ways: by clicking the button in the top-right corner of the interface, or by clicking the "To Station" button inside a marker popup.

To make cross-component toggling work, the application uses a shared service and standard property binding for the "to station button":

1. **Triggering the Change:** When a user clicks the button inside a marker popup, it sends a value of true to the body service to hide the map
2. **Passing the State:** The body component is subscribed to the observable and can read out the newly emitted value

This property instantly hides or reveals the map layout across the entire application interface.



EVERDONGENLAAN 31
2300 TURNHOUT
P: +32 (0)14 33 64 00
DOVERFUELINGSOLUTIONS.COM

6.2 ADDING CONTROLS

Leaflet controls are UI elements overlaying the map (like zoom buttons or scale bars) that allow user interaction.

6.2.1 IMPLEMENTING FULLSCREEN CONTROL

To add a fullscreen toggle button to your map, follow these two steps:

Import the Plugin

Import the fullscreen module at the top of your component file.

```
import FullScreen from 'leaflet.fullscreen';
```

Register the Control

Instantiate the control and inject it into your map object. You can define a title to set a tooltip.

```
this.m_map.addControl(new FullScreen({  
  title: "Fullscreen Button"  
}));
```

6.2.2 ADDING A CONTROL FOR CHANGING TILE LAYERS

This section explains how to implement tile layer picker control, allowing users to toggle between different map themes.

The application utilizes four background tile providers. Each configuration specifies its tiles server URL, maximum zoom boundaries, and mandatory legal attribution strings.

```
public m_osm =  
L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {  
    maxZoom: 19,  
    attribution: '© OpenStreetMap | Icons by <a  
href="https://www.freepik.com/" target="_blank">Freepik</a>  
});  
  
public m_openFreeMap = L.maplibreGL({  
    style: 'https://tiles.openfreemap.org/styles/liberty'  
});
```

Creating an object holding all tile providers

To populate the UI control panel, group the defined tile layers into a single baseMaps key-value object literal. The keys are dynamically translated using the localization service to display user-friendly names in the UI.

```
public baseMaps = {  
    "Default Theme": this.m_osm,  
    "Dark Theme": this.m_Stadia_AlidadeSmoothDark,  
    "Light Theme": this.m_CartoDB_Positron,  
    "OpenfreeMap Theme": this.m_openFreeMap  
};
```

Registering the Control on the Map

To display the selection menu on your map, pass the object holding the tile provider configuration into Leaflet's built-in layers control and add it to your map instance.

```
L.control.layers(this.baseMaps).addTo(this.m_map);
```

6.2.3 IMPLEMENTING A CUSTOM TOGGLE CLUSTERING BUTTON

You can extend the native `L.Control` class to build custom interactive buttons. Below is the implementation for the custom button that toggles marker clustering on and off found in the `initMap` method.

Extending `L.Control` and Setting Position

```
const ToggleClusteringButton = L.Control.extend({  
  options: {  
    position: 'topleft'  
  },  
});
```

`L.Control.extend`: Creates a new custom control class based on Leaflet's built-in `L.Control` class.

`position: 'topleft'`: Dictates where the control will anchor on the map layout.

`onAdd` Lifecycle Hook

```
onAdd: () => {}
```

`onAdd`: This lifecycle method fires automatically when `this.m_map.addControl()` is executed. It must build the HTML elements for the control and return the root container element.

Creating DOM Elements

```
const m_container = L.DomUtil.create('div', 'leaflet-bar leaflet-control');  
const m_button = L.DomUtil.create('a', 'toggle-clustering-button', m_container);
```

`L.DomUtil.create`: A Leaflet utility method that generates HTML elements.

`m_container`: Creates a parent `div`. The classes `'leaflet-bar leaflet-control'` are native Leaflet classes that give the button its standard map UI border, shadow, and background.

`m_button`: Creates an anchor (`<a>`) tag inside (`m_container`) to act as the clickable button.

Configuring toggle button with tooltip

```
m_button.innerHTML = this.m_defaultIconToggleClustering[0];  
m_button.title = this.m_TranslateService.instant("TTLabel_Alert_Map_Toggle_Clustering");  
m_button.style.cursor = 'pointer';  
m_button.style.fontSize = '20px';
```

`innerHTML`: Injects the initial FontAwesome icon graphic into the button.

Disabling map interaction

```
L.DomEvent.disableClickPropagation(m_button);
```

disableClickPropagation: This disables clicks registering on the map when clicking the button. With this off, it could be possible to click the toggle button and also clicking a station marker or cluster on the map.

Handling Click Events and Updating Icons

```
L.DomEvent.on(m_button, 'click', (e) => {  
  L.DomEvent.stop(e);  
  this.ToggleClustering();  
  updateIcon();  
});
```

L.DomEvent.on: Attaches a click listener to the button element.

L.DomEvent.stop(e): Halts the native browser click action.

this.ToggleClustering(): Call to switch marker rendering mode.

updateIcon(): Switches the icon to display inside the button.

Adding the control to the map

```
this.m_map.addControl(new ToggleClusteringButton());
```

addControl (): Method to add a L.Control object to the map. Here we add our own defined ToggleClusteringButton Control

6.2.4 ADDING THE LEGEND CONTROL

This section explains how to implement a custom, collapsible map legend. The legend programmatically generates a list of alert levels, pairs them with their corresponding marker icons, and includes a toggle button to show or hide the content.

Initializing the Control Anchor

First, instantiate a new blank Leaflet control object and assign its layout position to the bottom-right corner of the map.

```
const legend = new L.Control({ position: 'bottomright' });
```

The onAdd Lifecycle Hook and Container Creation

The onAdd method triggers automatically when the legend is appended to the map. Inside this hook, use L.DomUtil.create to generate a base div element with the CSS class 'legend'.

```
legend.onAdd = () => {  
  const div = L.DomUtil.create('div', 'legend');
```

Defining Localized Alert Labels

Create an array of strings containing the text labels for each alert status. These labels are retrieved dynamically using the translation service.

```
const alertLevels = [  
  this.m_TranslateService.instant('TTLabel_Filter_Unkonwn'),  
  this.m_TranslateService.instant('TTLabel_Filter_OK'),  
  this.m_TranslateService.instant('TTLabel_Status_Warning'),  
  this.m_TranslateService.instant('TTLabel_Actionbar_Title_Error')  
];
```

Programmatically Generating Legend Rows

Loop through the available marker icons (this.m_markerIcons) to assemble the list items. For each iteration, the loop builds an HTML string containing the icon image source and its matching localized text label, separated by a colon.

```
let m_legendItems: string = "";  
  
for (let i = 1; i <= this.m_markerIcons.size; i++) {  
  m_legendItems +=  
    '<i>' +  
    + ': ' + alertLevels[i - 1] + '</i>' + '<br>';  
}
```

Injecting HTML Structures

Inject a toggle button element and a content container div holding the generated legend rows directly into the main control element using innerHTML.

```
div.innerHTML += '<button type="button" class="legend-toggle-btn">' +  
this.m_TranslateService.instant('TextFmsLegend') + '</button>' +  
'<div class="legend-content">' + m_legendItems + '</div>';
```

Selecting Elements and Setting Initial Collapsed State

Locate the newly created button and content container from the DOM using querySelector. The component starts in a collapsed state (`m_collapsed = true`), meaning the inner content display style is set to 'none' and the button displays a simple "+" symbol.

```
const btn = div.querySelector('.legend-toggle-btn') as HTMLElement;  
const content = div.querySelector('.legend-content') as HTMLElement;  
  
let m_collapsed = true;  
content.style.display = m_collapsed ? 'none' : 'block';  
btn.innerHTML = m_collapsed ? '+' : '-' + this.m_TranslateService.instant('TextFmsLegend');
```

Handling the Click Interaction

Attach a native onclick listener to the button. Clicking the button toggles the `m_collapsed` boolean value, switches the content display between 'none' and 'block', and dynamically updates the button text label.

```
btn.onclick = () => {  
  m_collapsed = !m_collapsed;  
  
  content.style.display = m_collapsed ? 'none' : 'block';  
  btn.innerHTML = m_collapsed ? '+' : '-' +  
this.m_TranslateService.instant('TextFmsLegend');  
};  
return div;  
};
```

Mounting the Control

Finally, register the fully configured legend instance onto your active Leaflet map object.

```
this.m_map.addControl(legend);
```

6.3 CREATE MARKERS METHOD

This method processes a list of station data that comes in when selecting a CouCom in the tree, automatically adjusts the map boundaries to fit all stations on screen, creates stylized markers, and puts them on the map using either standard rendering or clustering.

Syncing with the Tree

Before creating markers, the method fetches the currently selected node from the tree service.

```
this.m_TreeService.getObservableTree().pipe(take(1))
  .subscribe(s => {
    if (s.m_eType == ETreeNodeType.Network || s.m_eType === ETreeNodeType.Station
    || s.m_eType === ETreeNodeType.Device) {
      this.m_networkNode = s;
      this.updateMapTitle();
    }
  });
```

m_networkNode: The network node is updated. This is needed to display the correct CouCom title on the map. Later this tree node is also injected into the Popup component. There the “To Station” button sends the tree node to the tree service to update the m_popupSelect\$ observable. This is to be able to programmatically walk the tree when the button is clicked to find the correct station.

updateMapTitle(): Updates the title text overlaying the map with the CouCom name

Formatting Coordinates

This method serves to create an array holding just the longitude and latitude to make it easier to create the markers later. The coordinates are extracted from the stations array holding all the Mapdata.

```
const m_coordinates: number[][] = []
stations.forEach(station => {
  if (station.GpsLatitude !== "" || station.GpsLongitude !== "") {
    const m_longlat: number[] = []
    m_longlat.push(parseFloat(station.GpsLatitude))
    m_longlat.push(parseFloat(station.GpsLongitude))
    m_coordinates.push(m_longlat)
  }
})
```

Framing the Map View (Auto-Bounding)

Rather than using a fixed, hardcoded center coordinate, or median position, the `setViewToBounds` method passes all coordinates to calculate a rectangular frame to set the view to. After setting the view, the map is locked to the calculated boundaries. It is not possible to pan outside the boundaries. Padding is foreseen for the boundaries to not obstruct animations when clicking a marker or CouCom.

```
this.setViewToBounds(m_coordinates)
```

`setViewToBounds()`: uses the `L.latLngBounds()` and `map.fitBounds()`. This ensures that all station markers are perfectly framed inside the screen when selecting a coucom and creating the markers. This also prevents redundant tiles being loaded

`setView()`: This helper method sets the view to the provided bounds. It also checks if an animation is already in progress, otherwise the map stutters when quickly selecting CouComs. After setting the view, the minimum and maximum zoom levels are applied on the map to prevent redundant tiles being loaded.

Defining Tooltip

Inside the main loop, an HTML string template is declared for the marker hover tooltips.

```
const m_tooltip = `  
  <kendo-popup [anchor]="anchor">  
    <div> ${stations[i].StationName} </div>  
  </kendo-popup>  
`
```

`.bindTooltip(m_tooltip, { sticky: true })`: Binds the tooltip to the marker, sticky means the tooltip stays under the cursor.

Dynamic Marker Icons

The method generates an icon using a lookup dictionary map (`m_markerIcons`) to retrieve an image file corresponding to the correct station alert level.

```
const m_markerIcon = L.icon({  
  iconUrl: this.m_markerIcons.get(stations[i].AlertLevel) || "",  
  iconSize: [35, 35],  
  iconAnchor: [25, 25],  
  popupAnchor: [-10, -25],  
});
```

`IconAnchor`: The exact pixel point of the icon image that pins to the map's coordinates (calculated from the top-left corner). For example, if your icon is a pin, this should be set to its sharp bottom tip so it points accurately.

`PopupAnchor`: The pixel coordinate where the popup bubble opens. This offset is calculated **relative to the `iconAnchor`**, allowing you to use negative values to float the popup perfectly above your icon.

Instantiating the Marker and Binding Event Handlers

If the coordinates are valid numeric values, a new L.marker instance is instantiated with custom user-accessibility options and native event handlers.

```
const marker = L.marker([m_coordinates[i][0], m_coordinates[i][1]], {  
  icon: m_markerIcon,  
  riseOnHover: true,  
  keyboard: true,  
  autoPanOnFocus: true,  
  title: this.m_alertLevelName.get(stations[i].AlertLevel)  
})  
.bindTooltip(m_tooltip, { sticky: true })  
.addEventListener("popupopen", (e: L.LeafletEvent) => { map.flyTo(e.target.getLatLng()); })  
.on('click', () => {  
  this.attachPopup(marker, stations[i]);  
});  
  
this.m_markerArray.push(marker);
```

Accessibility Properties: riseOnHover brings the icon to the front row when hovered. keyboard and autoPanOnFocus enable keyboard navigation support.

BindTooltip: Mounts the HTML string tooltip. The option { sticky: true } forces the tooltip bubble to follow the user's mouse cursor.

Popupopen Listener: Smoothly glides and centers the map view onto the selected marker using map.flyTo() when its popup opens.

Click Listener: listens to the user click event to run a programmatic layout attachment routine (attachPopup). This makes the popups lazy loaded, which improves the performance of the map.

m_markerArray: All the markers are pushed to this array to later add them all at once to the map rather than creating and adding them one by one.

Adding the markers to a layer: Layer Mounting vs Clustering

Once all markers are built and collected inside `m_markerArray`, the application evaluates the `m_isMarkerClustering` preference to decide how to add the markers to the map.

```
/** LayerGroup to which all the markers get added from the array*/  
public m_markerLayer!: L.LayerGroup;
```

```
/** Markerclustergroup to which all the markers get added in bulk from the array*/  
public m_donutCluster!: L.MarkerClusterGroup;
```

```
if (this.m_isMarkerClustering) {  
    this.m_donutCluster.addLayers(this.m_markerArray).addTo(this.m_map);  
} else {  
    this.m_markerArray.forEach(marker => marker.addTo(this.m_markerLayer))  
    this.m_markerLayer.addTo(this.m_map);  
}
```

Clustering Path: Feeds the markers into a new layer of a cluster sub-component (`m_donutCluster`) that is then added to the map.

Standard Path: Loops through the markers to drop each marker onto a flat, non-clustered map vector layer (`m_markerLayer`). Next, the layer itself, holding all the markers, is added to the map

6.4 COUPLING CUSTOM COMPONENTS TO POPUPS

The attachPopup method dynamically compiles a component (PopupComponent) and links it to a Leaflet marker popup at runtime.

Check for an Existing Popup

Before creating anything new, the method checks if the marker already has an initialized popup attached. If it does, Leaflet simply opens it and stops execution.

```
if (marker.getPopup()) {  
    marker.openPopup();  
    return;  
}
```

Programmatically Instantiate the Component

If no popup exists, Angular's createComponent routine is used to instantiate the PopupComponent out-of-context, passing the required dependency injector.

```
const componentRef = createComponent(PopupComponent, {  
    environmentInjector: this.m_Injector  
});
```

Inject Component Data Properties

The method feeds the necessary data directly into the created component instance. It passes the specific station data and the network tree node.

Since this component is created programmatically rather than via an HTML template, you cannot use declarative template data-binding. Assigning values **directly** to the **instance** properties achieves the exact same result as using **@Input()** variables on a standard component.

```
componentRef.instance.m_data = station;  
componentRef.instance.m_networkTreeNode = this.m_networkNode;
```

Attach Component to Application Lifecycle

To enable change detection and data-binding inside the custom component, its host view must be manually appended to the global framework application lifecycle loop.

Because this component is generated dynamically, the framework does not track it automatically. Attaching the view to the application reference ensures that data-binding, variables, and UI updates work properly inside the popup.

```
this.m_AppRef.attachView(componentRef.hostView);
```

Bind and Display the DOM Element

Leaflet needs a standard HTML element to display inside its popup bubble. We extract the component's raw HTML structure (`nativeElement`), hand it over to Leaflet's `bindPopup` function, and immediately display it on the map.

```
marker.bindPopup(componentRef.location.nativeElement);  
marker.openPopup();
```

Clean Up on Close (Memory Leak Prevention)

When a user closes the popup bubble, a listener executes a teardown routine to clean up system memory. It detaches the view from change detection, destroys the component instance entirely, and unbinds the popup from the marker.

```
marker.on('popupclose', () => {  
  this.m_AppRef.detachView(componentRef.hostView);  
  componentRef.destroy();  
  marker.unbindPopup();  
});
```

6.5 IMPLEMENTING DONUT CLUSTERS (CREATEDONUTCLUSTER)

This helper method instantiates a marker clustering group. Instead of standard circles, it displays a pie-chart-style donut graph that breaks down the alert states of the hidden markers inside that cluster.

Loading Labels

The method begins by fetching translated text strings for the various alert statuses. These strings are important because the clustering plugin relies on exact string matching to categorize the markers.

```
const m_arcTitleOk: string =  
this.m_TranslateService.instant('TTLabel_Filter_OK');  
const m_arcTitleWarning: string =  
this.m_TranslateService.instant('TTLabel_Status_Warning');  
const m_arcTitleError: string =  
this.m_TranslateService.instant('TTLabel_Actionbar_Title_Error');  
const m_arcTitleUndefined: string =  
this.m_TranslateService.instant('TTLabel_Filter_Unkonwn');
```

Bypassing Build Minification (Compiler Fix)

A technical hurdle when using Leaflet plugins in modern production environments is code minification. The build optimizer renames variables (like shrinking L down to a different character), which prevents third-party plugins from being recognized.

To bypass this, we force access to the global window scope using string notations. This safeguards our methods against rename optimization rules during production builds.

```
const leaflet = (window as any)['L'];  
  
return leaflet['DonutCluster'](...);
```

Configuring Core Map Behavior

The first configuration argument controls how the cluster group interacts with the map view:

```
{  
  showCoverageOnHover: true,  
  maxClusterRadius: 100,  
  disableClusteringAtZoom: 15  
}
```

showCoverageOnHover: Mousing over a cluster outlines the area covered by its internal markers.

maxClusterRadius: The pixel radius limit from a central marker. Higher numbers mean fewer, larger clusters. Lower numbers mean more, smaller clusters.

disableClusteringAtZoom: Automatically unpacks all clusters when zooming past this specific zoom level.

Setting Up Pie Chart Data Categories

The second argument dictates how data is split into segments inside the visual donut chart. The component scans the title property of incoming markers to calculate totals and match them against chart segments.

```
{  
  key: "title",  
  textContent: 'total',  
  legendClassName: 'donut-legend',  
  order: ["OK", "Warning", "Error", "Unknown"],  
  title: ['OK', 'Warning', 'Error', 'Unknown'],  
}
```

key: ' ': A marker must have the title option set to the key in order to be added to the cluster

textContent: ' ': Show the total number of items inside the center of the ring

legendClassName: ' ': The custom CSS class applied for legend styling

order: []: The drawing sequence of the chart slices

title: []: Segment labels that incoming marker titles must match

Assigning Segment Colors and Visuals

Finally, the method maps background colors to the translated text labels and applies dynamic sizing properties depending on how densely populated the cluster is.

```
arcColorDict: {  
  [m_arcTitleOk]: "#22b14c",    // Green for operational status  
  [m_arcTitleWarning]: "#f2bf0d", // Yellow for alerts  
  [m_arcTitleError]: "#f44336",  // Red for systematic failures  
  [m_arcTitleUndefined]: "#c3c3c3" // Gray for untracked statuses  
},  
style: (points) => ({  
  size: points.length < 10 ? 60 : 70, // Renders smaller clusters at 60px, and  
dense clusters (>10) at 70px  
  weight: 10,                        // Thickness of the outer ring  
  opacity: 1,                        // Ring transparency layer  
  fill: "#3e404d"                    // Background fill color for the inner counter circle  
}),  
}
```

6.6 TILE SERVICE FAILSAFE (SETTILEFAILSAFE)

This method acts as a backup for the map. If a tile provider goes offline or fails to load, the map automatically switches to the backup provider (OpenFreeMap) and notifies the user.

Setting the Error Threshold

The method tracks loading errors using a counter. To prevent switching prematurely due to a single slow image, it uses a threshold of 8 failed tile requests before triggering the backup.

```
let m_errorCount = 0;  
const m_maxErrors = 8;
```

Listening for Failures

The system listens to the active tile layer for the tileerror event. Every time a tile fails to load, the error counter increases by one.

```
tileLayer.on('tileerror', () => {  
  m_errorCount++;
```

Triggering the Switch

Once the failures reach the limit, the method checks if the broken layer is still visible on the map. If it is, Leaflet removes the broken layer and immediately adds the m_openFreeMap layer as a replacement. After that an alert is shown for 5 seconds to notify the user of the switch.

```
if (m_errorCount >= m_maxErrors) {  
  if (this.m_map.hasLayer(tileLayer)) {  
    this.m_map.removeLayer(tileLayer);  
    this.m_openFreeMap.addTo(this.m_map);
```


7. POPUP COMPONENT

This section documents the PopupComponent, which is a standalone component dynamically instantiated and bound inside the Leaflet map popups.

Component

The component is built as a **standalone** view element. This is because Leaflet operates outside of Angular's standard template rendering engine. This is why the popup must be compiled programmatically. Making the component standalone isolates its dependencies. The component explicitly declares its own imports (like Kendo and Translate modules), ensuring it has everything it needs to render successfully when injected directly into the DOM by Leaflet.

Input Data Properties

Although these fields are decorated as `@Input()`, they are not populated via standard HTML parent-child property bindings. Instead, their values are injected directly into the component instance by the map controller when a marker click opens the popup.

To station button (onButtonClick)

When a user clicks the “To Station” button, the station data and network tree node are sent to the tree service, where it will update the `m_popupSelect` observable. The tree component then subscribes to then find the station in the tree. Upon clicking the button, the view is also toggled by sending a value of “true” to the body service.

```
public onButtonClick(): void {  
  if (this.m_networkTreeNode || this.m_data) {  
    this.m_TreeService.popupSelect(this.m_networkTreeNode, this.m_data);  
    this.m_BodyService.toggleMap(true);  
  }  
}
```

8. TREE COMPONENT

8.1 WALKING THE TREE

When a user clicks the “To Station” button inside a station marker popup, the application must automatically find, expand, select, and scroll to that specific station inside the sidebar tree view. This section explains the logic that handles the tree walking.

Listening to Popup Click Events

The tree component subscribes to an observable stream inside the tree service. When a popup button is clicked, this stream emits an array containing two critical pieces of information: the parent network tree node and the specific station data.

```
this.m_PopupSubscription = this.m_TreeService.m_popupSelect$.pipe(  
  switchMap(s => {  
    currentTreeNode = s[0]; // Network Node  
    mapData = s[1];        // Station Data  
  })  
);
```

switchMap: Automatically cancels any previous pending tree-walking requests if the user rapidly clicks different stations on the map.

currentTreeNode: When a Coucom was selected, the current tree node is a network. When a station was selected in the left tree, the current tree node is a station. Also when a device was selected in the left tree, the current tree node is a device.

mapData: Holds all the station data of the corresponding clicked popup.

Guarding Against Duplicate Actions

The method checks if the requested station is already selected or invalid. If nothing has changed, it cancels the operation. Otherwise, it updates its tracking variables. The index and tree node are saved. This is needed for when a user closes the map and switches network in the left tree. In that case the network is lost and the saved network can be used.

```
if (currentTreeNode == undefined) {  
  currentTreeNode = lastNetworkTreeNode;  
}  
if (currentTreeNode.m_eType == 0 || mapData.Index == lastTreeNodeIndex &&  
this.isSelected(foundTreenode)) {  
  return EMPTY;  
} else {  
  lastTreeNodeIndex = mapData.Index;  
  lastNetworkTreeNode = currentTreeNode;  
}
```

Finding and Expanding the Network Node

If the user clicked a station, the current tree node needs to be a network node. So in case the current tree node holds a station, then the node is walked up 2 levels by accessing the parentNode. When it is certain that the current node is a network, then the node is expanded by calling the tree service with nodesExpandedAdd.

```
if (currentTreeNode.m_eType == 6) { // Type 6 is a station node
    currentTreeNode = currentTreeNode.parentNode.parentNode;
}

if (!this.isExpanded(currentTreeNode) && currentTreeNode.m_eType == 3) {
    this.m_TreeService.nodesExpandedAdd(currentTreeNode);
}
```

Navigating to the Stations Subfolder

Inside the expanded network, the code looks at the child nodes to locate the "Stations" folder (s[1]). It expands this subfolder so the individual stations inside it become visible in the DOM.

```
return this.m_TreeView.children(currentTreeNode).pipe(
    map(s => {
        currentTreeNode = s[1] as TreeNode;

        if (!this.isExpanded(currentTreeNode)) {
            this.m_TreeService.nodesExpandedAdd(currentTreeNode);
        }
    })
);
```

Finding the Exact Station

Because the UI elements were just expanded, the code forces Angular to refresh using detectChanges(). It then searches the tree hierarchy using the unique station number to locate the exact node. Once found, it marks it as selected.

```
this.m_ChangeDetector.detectChanges();

rootNode = this.m_TreeView.itemLookup(this.m_NetworkIndex);

if (rootNode) {
    foundTreenode = this.searchTreeNode(mapData.StationNumber, rootNode);
}

if (foundTreenode) {
    this.m_TreeService.setSelectedTreeNode(foundTreenode);
}
```

Focusing and Scrolling the View

Finally, the system targets the specific tree index to set the keyboard focus. A short `setTimeout` delay runs right after to allow the browser layout to settle, smoothly scrolling to the highlighted station into the center of the sidebar view. Then the tree is scrolled to the left, otherwise the tree window is scrolled horizontally, hiding the network nodes.

The animations of the kendo tree were disabled to make searching the tree possible. Otherwise, the silent animations would disrupt the workflow and the tree search would fail because the DOM was lagging behind.

```
this.m_TreeView.focus(this.m_NetworkIndex + "_1_" + mapData.Index);

setTimeout(() => {
  const focusElem = document.querySelector('.k-focus');
  if (focusElem) {
    focusElem.scrollIntoView({ block: "center", inline: "start" });
    document.querySelector('.scrollContainer').scrollLeft -= 5000;
  }
}, 0);
```

8.2 RECURSIVE TREE SEARCHING (SEARCHTREENODE)

When navigating the Tree, finding a specific station requires searching through multiple nested nodes (station groups and stations). The `searchTreeNode` helper method solves this by using a recursive search to scan all layers of the tree hierarchy until it matches the required station number.

Input Validation Guard

The method first checks if the target station number `lookupString` is valid. If the string is undefined or null, it stops execution. This is to guard against when the map is first opened up when nothing is selected yet.

```
if (('undefined' === typeof (lookupString)) || (null === lookupString)) {
  return;
}
```

Iterating Child Nodes

The method loops through the immediate children of the provided `parentNode`. For each child, it extracts the underlying data object and casts it to a `TreeNode` type so its station properties can be read.

```
parentNode.children.forEach((childNode: TreeItemLookup) => {
  const treeIndex = childNode.item.dataItem as TreeNode;
```

Match Verification and Terminating Iteration

It compares the target station number against the current child node's station number string. If they match, it stores the node in the property this.m_FoundTreeNode and breaks out of the current forEach loop.

```
if (lookupString === treeIndex.m_Station.m_strStationNumber) {  
    bFound = true;  
    this.m_FoundTreeNode = childNode.item.dataItem as TreeNode;  
    return; // Exits the current 'forEach' layer  
}
```

Recursive Search

If no match is found at the current level, the method calls itself (this.searchTreeNode) using the current childNode as the *new parent*. This forces the logic to dive a layer deeper into the tree structure, continuing until the station is found or all branches have been scanned.

```
if (!bFound) {  
    this.searchTreeNode(lookupString, childNode);  
}
```

8.3 HANDLING USER SELECTION CHANGES (ONSELECTIONCHANGE)

This method fires automatically whenever a user manually clicks a node inside the sidebar tree view. It updates the state, fetches new station data from the API in the cup-alerts service based on the selected node, and automatically switches the layout view if a specific station or device is selected.

Selecting a Network Node

If the user clicks a Network node, the system stores its index and fetches all associated stations for that network, combining them with any active user filters.

```
if (treeNode.m_eType === ETreeNodeType.Network) {  
  this.m_NetworkIndex = selected.index;  
  this.m_ApiService.getStations(  
    this.m_TreeService.combineFilterAndTreeNode(this.m_Filter, treeNode),  
    true, true  
  ).subscribe();  
}
```

Selecting a Station Node

If a **Station** node is selected, the application automatically hides the map view to display the main dashboard tabs. It also extracts the parent network's index using a string split, ensuring that the API request targets the parent network node.

```
if (treeNode.m_eType === ETreeNodeType.Station) {  
  this.m_BodyService.toggleMap(true); // Close map, show dashboard  
  this.m_NetworkIndex = selected.index.split('_').slice(0, 2).join('_');  
  this.m_ApiService.getStations(  
    this.m_TreeService.combineFilterAndTreeNode(this.m_Filter,  
    treeNode.parentNode.parentNode),  
    true, true  
  ).subscribe();  
}
```

Index Splitting Explained: Kendo tree indices use underscores for nesting (e.g., "1_0_3"). By splitting the string and slicing the first two parts ("1_0"), the system strips away the station identifier and extracts the exact index of the parent network.

Getstations: The cup-alerts service is called to fetch new station data. This method is extended with a boolean parameter "sendToMap" to determine if the retrieved data needs to be sent to the map component

Selecting a Device Node

If a **Device** node is clicked, it triggers the same layout toggle behavior. However, because a device lives one layer deeper than a station, the system navigates three levels up (`parentNode.parentNode.parentNode`) to find the root network and fetch the correct station overview data.

```
if (treeNode.m_eType === ETreeNodeType.Device) {  
  this.m_BodyService.toggleMap(true);  
  this.m_NetworkIndex = selected.index.split('_').slice(0, 2).join('_');  
  this.m_ApiService.getStations(  
    this.m_TreeService.combineFilterAndTreeNode(this.m_Filter,  
    treeNode.parentNode.parentNode.parentNode),  
    true, true  
  ).subscribe();  
}
```

8.4 REFRESH TIMER (M_REFRESHTIMERSUBSCRIPTION)

This timer periodically refreshes the map and sidebar tree in the background to ensure data stays up to date without reloading the page.

Resetting Active Subscriptions

When the timer fires, it unsubscribes from existing tree streams to prevent memory leaks, then fetches the latest snapshot of the selected tree node.

```
this.m_TreeServiceSubscription?.unsubscribe();  
this.m_TreeServiceSubscription = this.m_TreeService.getObservableTree().subscribe(s  
=> {  
  treenode = s;  
});
```

Evaluating Refresh Conditions

If a valid Network node (eType == 3) is active, the system checks three conditions before calling the API:

```
if (!this.m_mapIsHidden && treenode.m_eType == 3 &&  
    !this.m_TreeService.isNodeExpanded(stationsNode)) {  
    this.m_ApiService.getStations(...).subscribe();  
}
```

Only if the map is visible (!this.m_mapIsHidden)

A network is selected (eType == 3).

The stations node is closed: *If the list were open, Kendo would refresh automatically. This failsafe handles updates specifically when the list is hidden but the map is visible.*

Re-rendering the Tree Layout

Finally, the system forces the Kendo component to re-fetch and re-draw its child nodes with the renewed data.

```
this.m_TreeView.rebindChildren();
```